OAP200V · OBJEKTORIENTERT ANALYSE, DESIGN OG PROGRAMMERING

LibriKeep

Private Book Library Manager

Prosjektrapport – problemanalyse, systemdesign og implementasjon

Emne	OAP200V – Objektorientert analyse, design og programmering, høst 2026
Institusjon	Universitetet i Sørøst-Norge (USN)
Faglærere	Katerina Mangaroska og Boban Vesin
Gruppemedlemmer	Stian Petersen, Santheep **[fyll inn: etternavn]**, Andreas **[fyll inn: etternavn]**, Eryk **[fyll inn: etternavn]**, Laura Gorska, Frank **[fyll inn: etternavn]**
Dato	12.10.2026
Status	Arbeidsversjon (applikasjon v0.1.1)

Innhold

1 Innledning 4

1.1 Bakgrunn og formål 4
1.2 Omfang og målgruppe 4
1.3 Teknologi og leveranser 4
1.4 Organisering og milepæler 4
1.5 Statusmarkering i rapporten 5

2 Problemanalyse 6

2.1 Problemstilling 6
2.2 Brukere og systemgrenser 6
2.3 Funksjonelle krav 6
2.4 Ikke-funksjonelle krav 7

3 Systemdesign 8

3.1 Use case-modell 8
3.2 Domenemodell og klassedesign 8
3.3 Sekvensdiagrammer 9
3.4 Arkitekturmodell (4+1) 11

4 Databasedesign 13

4.1 ER-diagram 13
4.2 De fire hovedentitetene 13
4.3 Integritetsregler og feilhåndtering 13
4.4 Testdata og testdatabase 14

5 Brukergrensesnittdesign 15

5.1 Designprosess 15
5.2 Implementert brukergrensesnitt (v0.1) 15
5.3 Navigasjonsflyt 15
5.4 Brukervennlighet og tilgjengelighet 16

6 Implementasjonsoversikt 17

6.1 Krav mot implementasjon 17
6.2 Viktige pakker og klasser 17
6.3 Sentrale mekanismer 17
6.4 JavaDoc og kodedokumentasjon 18
6.5 Bygg og kjøring 18

7 Testinnsikt 19

7.1 Teststrategi 19
7.2 Verktøy og miljø 19
7.3 Resultater fra automatiske tester 19
7.4 Hva testene dekker 19
7.5 Manuell funksjonstesting 20
7.6 Utfordringer i testarbeidet og løsninger 20

8 Utfordringer og begrensninger 21

8.1 Tekniske og konseptuelle utfordringer 21

8.2 Begrensninger i v0.1 21
8.3 Neste steg og mulige utvidelser 21
9 Individuelle bidrag 22
9.1 Arbeidsform 22
9.2 Bidragstabell 22
9.3 Individuelle refleksjoner 22
10 Konklusjon 24
Vedlegg A – Sjekkliste før innlevering 25
Vedlegg B – Sporbarhet mot emnets krav 26

1 Innledning

1.1 Bakgrunn og formål

Mange privatpersoner har bøker spredt over flere rom, hyller og esker. Over tid blir det vanskelig å vite hvilke bøker man eier, hvor de står, hvilke som er lest, og hvilke som hører til en bestemt forfatter eller sjanger. LibriKeep er gruppens løsning på eksamensoppgaven «Private Book Library Manager» i OAP200V (emnebeskrivelsen, kapittel 8): en desktop-applikasjon for å registrere, organisere og søke i en privat boksamling.

Rapporten dokumenterer hvordan gruppen har gått fra problemstilling til krav, UML-modeller, databasedesign og en kjørbar Java-applikasjon, og hvordan løsningen er testet. Den følger rapportstrukturen i emnebeskrivelsen (kapittel 7.6).

1.2 Omfang og målgruppe

Systemet er et **enbrukersystem** for eieren av samlingen. Gruppen besluttet 25.09.2026 å fjerne bruker- og innloggingsbegrepet fra datamodellen, siden en privat samling ikke trenger flere brukerkontoer. Omfanget følger nøkkelfunksjonene i oppgaveteksten (kapittel 8.1) og den foreslåtte databasen med fire tabeller (kapittel 8.2):

- **Implementert i v0.1.1:** registrering og vedlikehold av bøker, forfattere, sjangre og bokhyller, kobling av hver bok til forfatter, sjanger og bokhylle, lesestatus, detaljvisning, søk og filtrering, og oppsummeringsrapporter i backend.
- **Planlagt før innlevering (v0.2):** statistikkvisning i brukergrensesnittet basert på de eksisterende rapportene, og CSV-eksport.

Gruppen har bevisst holdt omfanget til oppgavens krav, slik faglærerne har anbefalt. Ideer utover dette er samlet som mulige utvidelser i kapittel 8.3.

1.3 Teknologi og leveranser

Tabell *1. Teknisk plattform for LibriKeep v0.1.1.*

Område	Valg
Språk og plattform	Java 17 (Java SE)
Brukergrensesnitt	JavaFX 21
Database	MySQL 8 via JDBC (mysql-connector-j); SQL-skript for oppretting og testdata i `database/`
Bygg og avhengigheter	Maven (pom.xml); prosjektet importeres i Eclipse som Maven-prosjekt
Testing	JUnit 5 (enhetstester og integrasjonstester mot MySQL 8.0)
Kildekode	GitHub: `Private-Library-Java-App-Org/Private-Library-Java-App` (versjon v0.1.1)
Leveranser (kap. 7.3.1)	Rapport (PDF); én ZIP med Eclipse-prosjekt, pom.xml, SQL-skript, generert JavaDoc i `docs/` og brukermanual; presentasjon til demo

1.4 Organisering og milepæler

Gruppen bruker GitHub til kildekode, rapport og oppgaver (Issues), Microsoft Planner til fordeling av arbeid og Teams til koordinering og deling av skisser. Hvert GitHub Issue tildeles én person, slik at individuelle bidrag kan spores (se kapittel 9).

Tabell *2. Milepæler og frister.*

Dato	Milepæl
17.09.2026	Arbeidsflyt etablert: GitHub Issues (15 stk.), Planner og Teams
25.09.2026	Datamodellen forenklet til én bruker
12.10.2026	Arbeidsversjon av rapporten; applikasjon v0.1.1
09.11 og 11.11.2026	Presentasjon av obligatorisk oppgave
11.11.2026 kl. 23:59	Obligatorisk innlevering
16.11.2026	Semesterslutt
Ikke fastsatt	Endelig innlevering og muntlig eksamen **[fyll inn: dato når den er kunngjort i Canvas]**

1.5 Statusmarkering i rapporten

Tabell *3. Statusbegreper.*

Status	Betydning
Implementert	Finnes i v0.1.1 av kodebasen og kan demonstreres
Implementert (backend)	Logikk og databaselag finnes og er testet, men har ennå ingen egen visning i brukergrensesnittet
Planlagt	Prioritert for v0.2 før innlevering; ikke implementert ennå

2 Problemanalyse

2.1 Problemstilling

Eieren av en privat boksamling trenger én samlet oversikt over hvilke bøker som finnes, hvor de står, hvem som har skrevet dem, hvilken sjanger de tilhører og om de er lest. I dag løses dette typisk manuelt eller fragmentert:

- Hukommelse og fysisk leting gir dobbeltkjøp og bøker som ikke blir funnet.
- Regneark mangler relasjoner og integritetsregler, for eksempel stavevarianter av samme forfatter og ingen kontroll av ISBN-duplikater.
- Generelle lese-apper er nettbaserte og kontobaserte, og har ingen modell for fysisk plassering (rom og hylle).

LibriKeep løser dette med en lokal desktop-applikasjon og en relasjonsdatabase der hver bok er knyttet til nøyaktig én forfatter, én hovedsjanger og én bokhylle, og der databasen håndhever reglene for unikhet og referanseintegritet.

2.2 Brukere og systemgrenser

Tabell *4. Brukere og systemgrenser.*

Aktør / system	Rolle
Eier av samlingen	Eneste bruker. Registrerer og vedlikeholder samlingen, søker, setter lesestatus og henter ut rapporter.
MySQL-database	Ekstern lagring av alle data; nås via JDBC.
Filsystem	Konfigurasjonsfil (`config.properties`) og planlagt CSV-eksport.

2.3 Funksjonelle krav

Kravene FR1–FR11 tilsvarer nøkkelfunksjonene i oppgaveteksten (kapittel 8.1) én til én. FR12–FR14 er støttefunksjoner som gruppen har valgt. Hvert krav har prioritet (Må/Bør/Kan), ansvarlig medlem og status per v0.1.1.

Tabell *5. Funksjonelle krav.*

ID	Krav	Prioritet	Ansvarlig	Status
FR1	Legge til, redigere og fjerne bøker	Må	Eryk	Implementert
FR2	Registrere og administrere forfattere	Må	Eryk	Implementert
FR3	Registrere og administrere sjangre	Må	Eryk	Implementert
FR4	Registrere og administrere bokhyller (navn, rom, hyllenummer)	Må	Eryk	Implementert
FR5	Knytte hver bok til én forfatter, én sjanger og én bokhylle	Må	Eryk / Stian	Implementert
FR6	Markere bøker som lest, ulest eller leser nå	Må	Eryk	Implementert
FR7	Vise detaljert informasjon om hver bok	Må	Eryk	Implementert
FR8	Søke etter bøker på tittel, forfatter, sjanger eller bokhylle (og filtrere på lesestatus)	Må	Eryk	Implementert
FR9	Vise alle bøker skrevet av en bestemt forfatter	Må	Eryk / Stian	Implementert
FR10	Vise alle bøker plassert på en bestemt bokhylle	Må	Eryk / Stian	Implementert
FR11	Oppsummeringsrapporter: bøker per sjanger, per forfatter, leste, uleste og totalt antall	Må	Stian (backend), Santheep (visning)	Implementert (backend); visning planlagt

ID	Krav	Prioritet	Ansvarlig	Status
FR12	Hindre sletting av forfatter, sjanger eller bokhylle som har bøker	Må	Eryk / Stian	Implementert
FR13	Statuslinje med sammendrag (totalt, lest, leser nå, ulest)	Bør	Eryk	Implementert
FR14	Eksportere samlingen og rapporter til CSV (lesing og skriving mot filsystemet)	Bør	Santheep	Planlagt

Ny nummerering: i tidligere Issues og diagrammer er rapportene omtalt som FR7, CSV-eksport som FR8 og sletteskjermingen som FR9. Disse heter nå FR11, FR14 og FR12. **[fyll inn: oppdater ID-er og ansvarlige i GitHub Issues]**

2.4 Ikke-funksjonelle krav

De ikke-funksjonelle kravene bygger på de tekniske rammene og vurderingskriteriene i emnebeskrivelsen (kapittel 7.2–7.4).

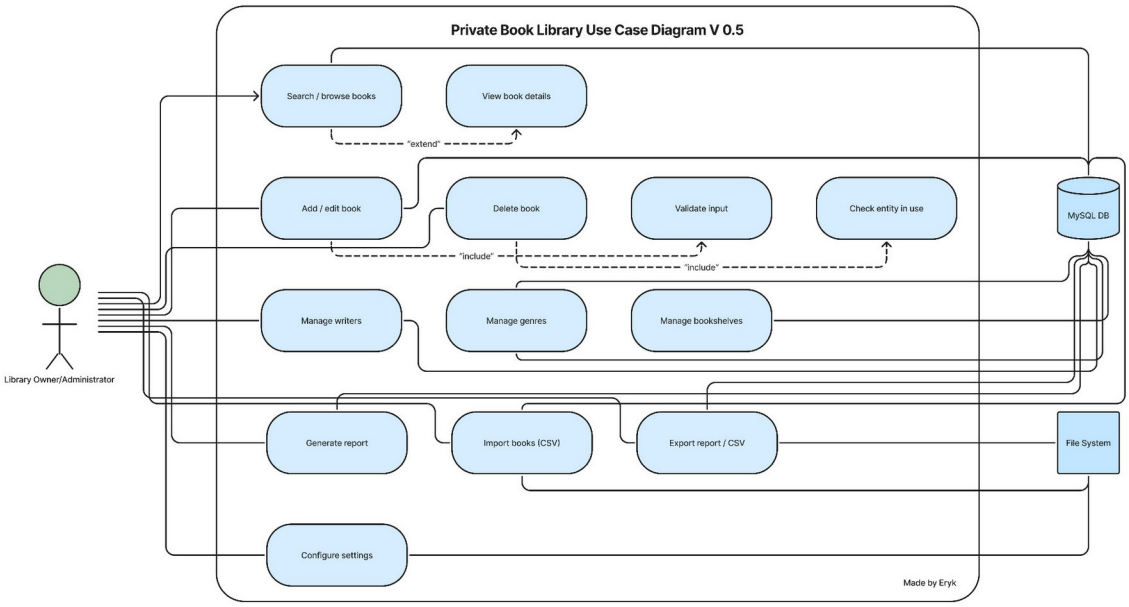
Tabell *6. Ikke-funksjonelle krav.*

ID	Kategori	Krav	Prioritet	Status / verifikasjon
NFR1	Teknologi	Koden skrives i Java SE (Java 17) med JavaFX 21 som GUI	Må	Oppfylt
NFR2	Persistens	Data lagres i MySQL 8 via JDBC; SQL-skript for oppretting og testdata leveres	Må	Oppfylt (`schema.sql`, `seed.sql`); verifisert med integrasjonstester
NFR3	Bygg	Alle eksterne avhengigheter håndteres av Maven; pom.xml leveres	Må	Oppfylt
NFR4	Portabilitet	ZIP-filen kan importeres i Eclipse på rotnivå uten tilpasning, testet på flere maskiner	Må	Ikke verifisert ennå (T07)
NFR5	Ressursfiler	Applikasjonen genererer nødvendige ressursfiler selv hvis de mangler	Må	Oppfylt: ConfigManager oppretter `config.properties`
NFR6	Dokumentasjon	Generert JavaDoc (HTML) leveres i egen `docs/`-mappe	Må	`mvn javadoc:javadoc` kjører uten feil; `docs/` i ZIP **[fyll inn]**
NFR7	Dokumentasjon	Hver klasse oppgir forfatter(e), bidrag, hensikt og forklaring av metodene	Må	Delvis; kontroll av alle klasser **[fyll inn]**
NFR8	Robusthet	Alle kjøretidsfeil håndteres: ugyldig inndata og databasefeil gir forståelig melding (Alert), ikke krasj	Må	Oppfylt via InputValidator, egne unntak og AlertUtil; manuell test T02/T06 **[fyll inn]**
NFR9	Kodekvalitet	Høy kohesjon og lav kobling på klasse- og pakkenivå, sterk innkapsling; klassemedlemmer samsvarer med klassediagrammet	Må	Lagdeling oppfylt; klassediagrammet må oppdateres (kap. 3.2)
NFR10	Dataintegritet	Unik ISBN, unik forfatter og unikt sjangernavn; ingen foreldreløse referanser; norske tegn lagres korrekt	Må	Oppfylt (UNIQUE, FK ON DELETE RESTRICT, utf8mb4); testet
NFR11	Sikkerhet	Databasepassord holdes utenfor Git; alle spørringer bruker PreparedStatement	Må	Oppfylt; SQL-injeksjon testet
NFR12	Brukervennlighet	Hovedvinduet gir tilgang til alle hovedfunksjoner; hver funksjon har egne UI-komponenter	Må	Delvis; brukertest **[fyll inn]**

3 Systemdesign

3.1 Use case-modell

Use case-diagrammet (v0.5) har én aktør, eieren av samlingen, og to eksterne systemer: MySQL-databasen og filsystemet. «Legg til/rediger bok» inkluderer validering av inndata, og «Slett bok» og administrasjon av forfattere, sjangre og bokhyller inkluderer kontroll av om enheten er i bruk.



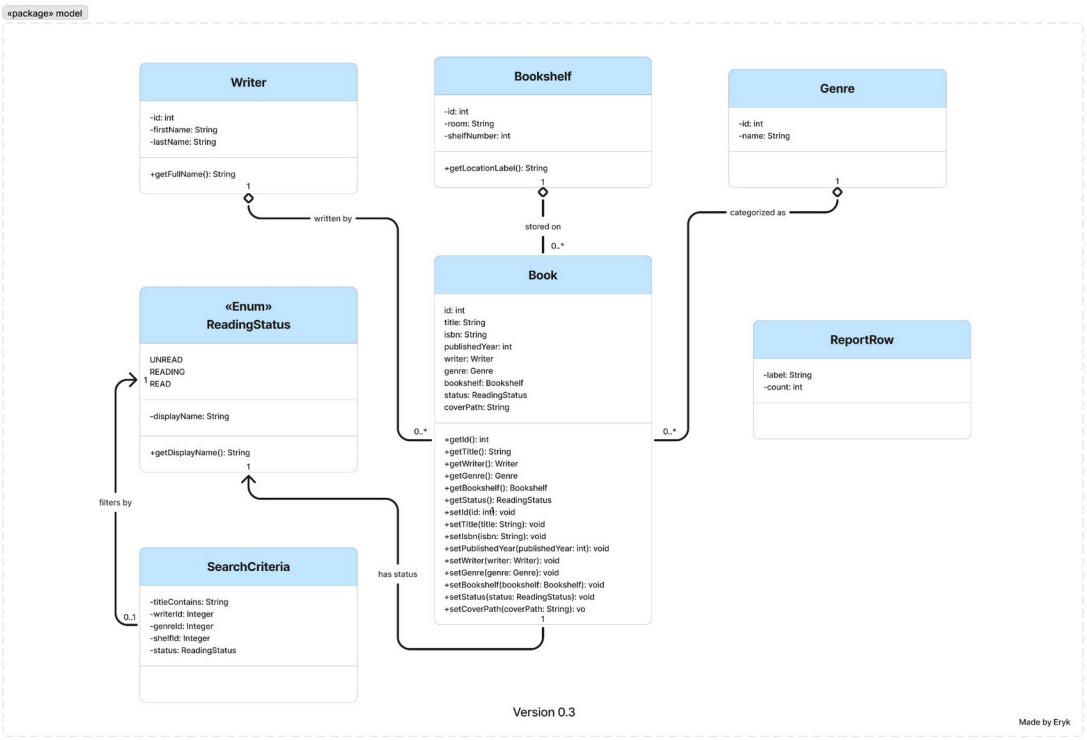
Figur *1. Use case-diagram for LibriKeep, versjon 0.5. CSV-import er vist i diagrammet, men ligger utenfor omfanget.*

Tabell *7. Hovedbrukstilfeller.*

UC	Brukstilfelle	Beskrivelse	Krav	Status
UC1	Søke og bla i bøker	Filtrere boklisten på tittel, forfatter, sjanger, bokhylle og lesestatus; utvides med «Vis bokdetaljer».	FR7–FR10	Implementert
UC2	Legge til / redigere bok	Fylle ut skjema, validere og lagre; inkluderer «Valider inndata».	FR1, FR5, FR6	Implementert
UC3	Slette bok	Bekrefte og slette en bok.	FR1	Implementert
UC4	Administrere forfattere, sjangre og bokhyller	CRUD; sletting inkluderer «Kontroller om enheten er i bruk».	FR2–FR4, FR12	Implementert
UC5	Generere rapport	Oppsummering per sjanger, forfatter, bokhylle og lesestatus, og totalt.	FR11	Backend implementert
UC6	Eksportere rapport / CSV	Skrive samlingen eller en rapport til fil.	FR14	Planlagt
UC7	Konfigurere innstillinger	Databasetilkobling via `config.properties`.	NFR5	Implementert

3.2 Domenemodell og klassedesign

Domenemodellen består av fire entiteter (Book, Writer, Genre, Bookshelf), enumtypen ReadingStatus (UNREAD, READING, READ) og hjelpeklassen SearchCriteria, som samler valgfrie filterkriterier. Book har referanser til nøyaktig én Writer, én Genre og én Bookshelf, som speiler fremmednøklene i databasen.



Figur *2. Klassediagram for model-pakken, versjon 0.3.*

Designvalg:

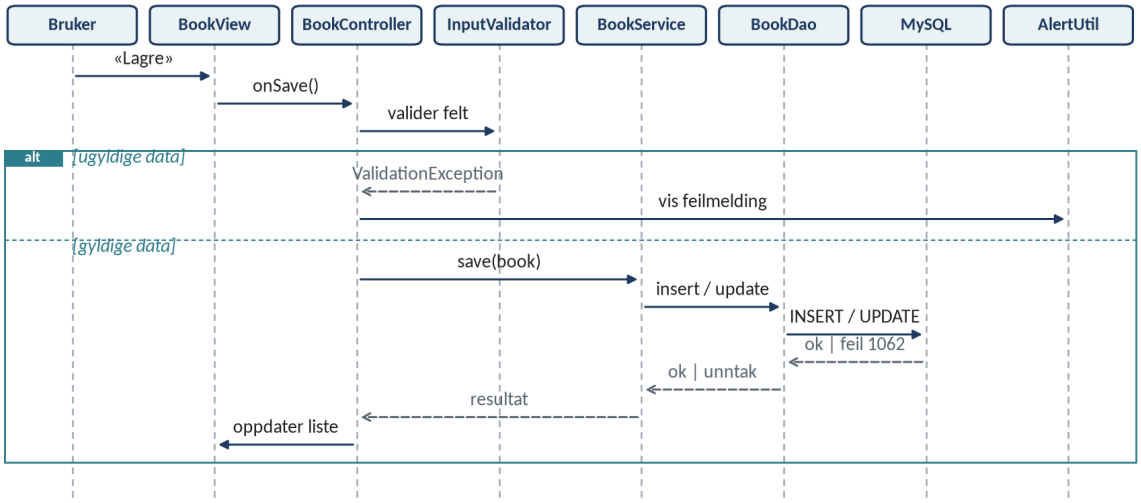
- **Innkapsling:** alle felt er private og eksponeres via get-/set-metoder; ReadingStatus har et visningsnavn for brukergrensesnittet.
- **Kohesjon:** modellklassene inneholder bare data og enkel avledet visning (f.eks. `getFullName()` og `getLocationLabel()`). SQL ligger kun i DAO-laget og forretningsregler i tjenestelaget.
- **Lav kobling:** tjenestene avhenger av DAO-grensesnitt (`BookDao`, `WriterDao` osv.), ikke av JDBC-implementasjonene. Det gjør det mulig å teste tjenestene med falske DAO-er.
- **Rapportmodell:** `ReportRow` i v0.3 er implementert som den uforanderlige klassen `ReportEntry` (id, navn, antall bøker), siden en rapport er et øyeblikksbilde.

Kjent avvik: koden har noen felt som ikke er med i v0.3, blant annet `nationality` og `birthYear` på Writer, `description` på Genre og `pageCount` på Book. Feltet `coverPath` i v0.3 hører til omslagsbilder, som ligger utenfor omfanget. Emnet krever at klassemedlemmene samsvarer med klassediagrammet. **[fyll inn: oppdater klassediagrammet til v0.4 slik at det samsvarer med koden]**

3.3 Sekvensdiagrammer

De tre diagrammene viser de sentrale forløpene: å lagre en bok, å slette en forfatter med sletteskjerming og å generere en rapport.

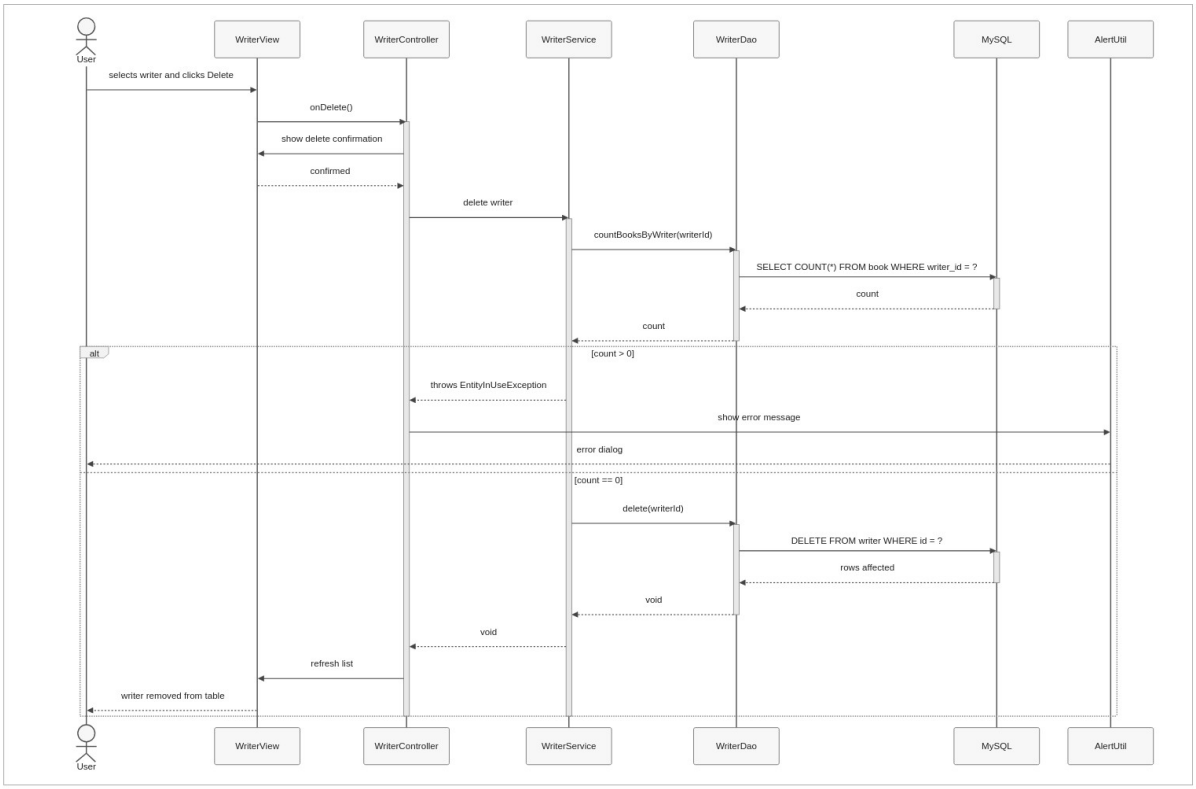
3.3.1 Lagre bok (FR1, FR5)



Figur *3. Sekvensdiagram: lagre bok med validering og duplikatkontroll.*

Inndata valideres i `InputValidator` før lagring. Et brudd på den unike ISBN-regelen gir MySQL-feil 1062, som `JdbcSupport` oversetter til `DuplicateEntryException`, og brukeren får en melding via AlertUtil. **[fyll inn: kontroller klassenavnene for view og controller mot koden]**

3.3.2 Slette forfatter med sletteskjerming (FR12)

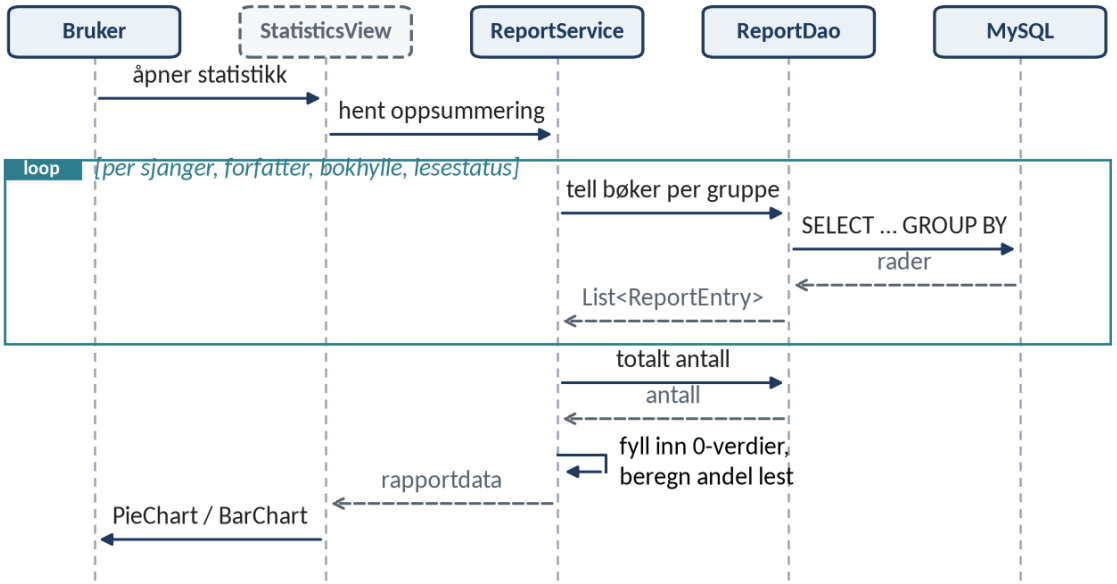


Figur *4. Sekvensdiagram: slette forfatter med kontroll av tilknyttede bøker (FR12, tidligere FR9).*

Controlleren ber om bekreftelse, og tjenesten kontrollerer om forfatteren har bøker. Hvis forfatteren er i bruk, kastes `EntityInUseException`, og brukeren får en feilmelding. I v0.1 håndheves regelen også i databasen: fremmednøklene har `ON DELETE RESTRICT`, og `JdbcSupport` oversetter MySQL-feil 1451 til

`EntityInUseException`. **[fyll inn: bekreft om WriterService teller bøker før sletting, slik diagrammet viser, eller om koden bare bruker databaseregelen]**

3.3.3 Generere oppsummeringsrapport (FR11)

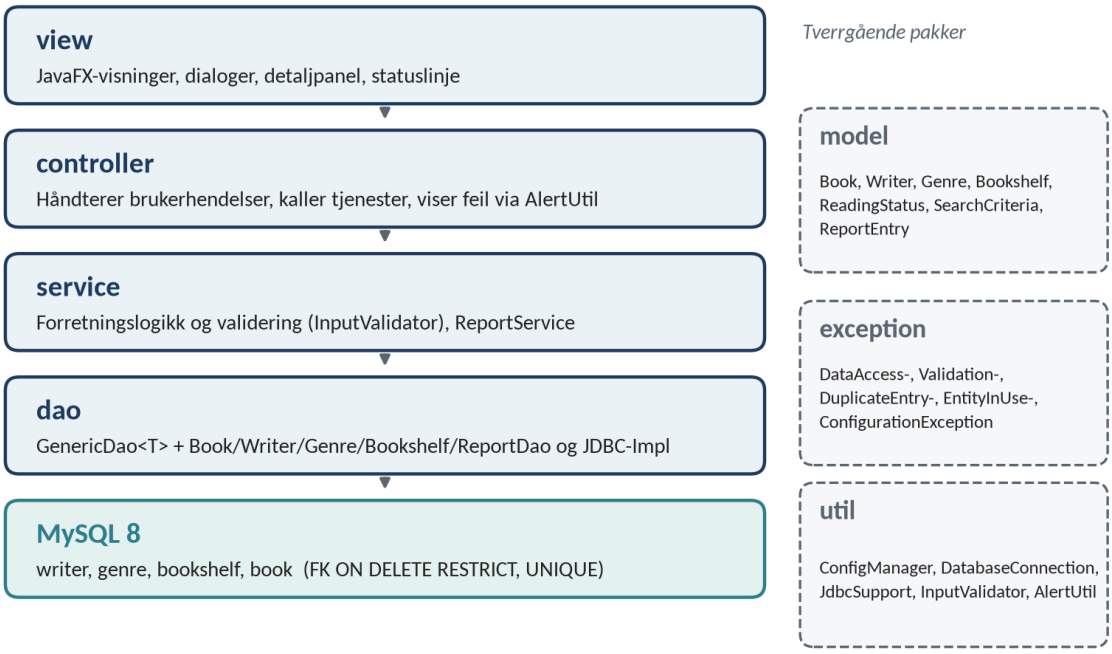


Figur *5. Sekvensdiagram: oppsummeringsrapport. StatisticsView (stiplet) er planlagt; ReportService og ReportDao er implementert.*

`ReportService` kaller `ReportDao` én gang per rapporttype. Hver spørring er én `GROUP BY` med `LEFT JOIN`, slik at grupper uten bøker vises med 0. Tjenesten fyller inn manglende lesestatuser med 0 og regner ut andelen leste bøker. Visningen med diagrammer er planlagt for v0.2.

3.4 Arkitekturmodell (4+1)

Applikasjonen er bygd som en lagdelt MVC-arkitektur. Avhengighetene går kun nedover, og pakkene model, exception og util brukes på tvers av lagene.



Figur *6. Logisk visning: lagdeling og pakker i LibriKeep v0.1.*

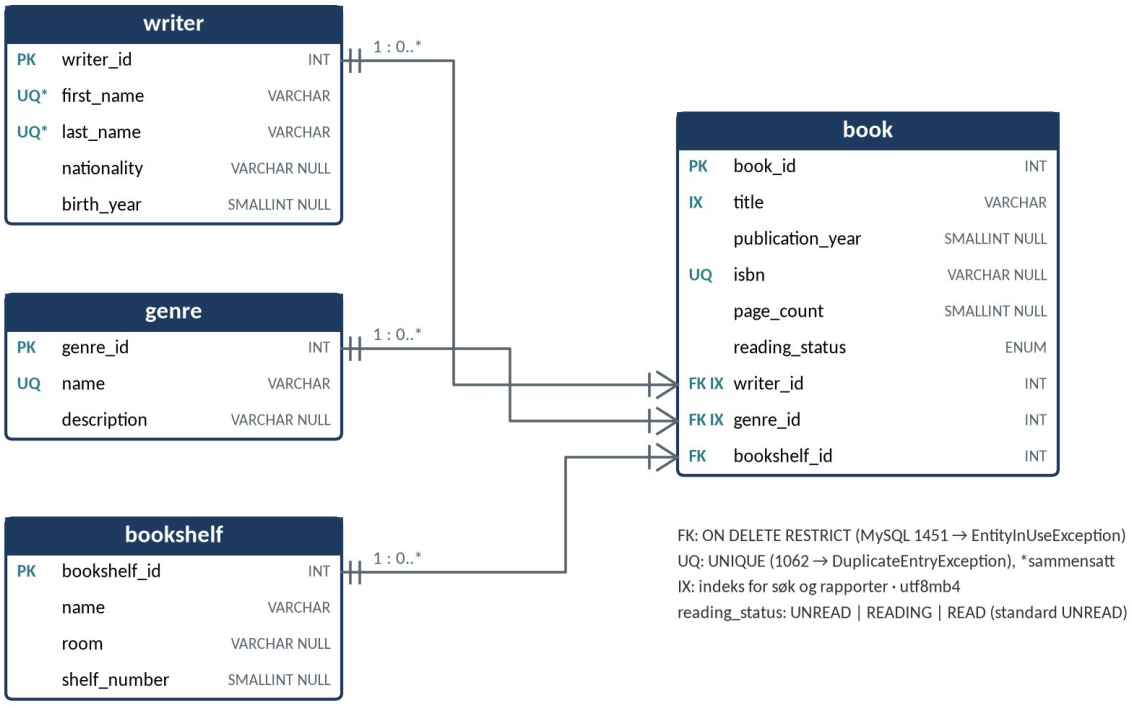
Tabell *8. 4+1-visninger for LibriKeep.*

Visning	Innhold
Logisk	Pakkene `view`, `controller`, `service`, `dao`, `model`, `util` og `exception` (figur 6). DAO-laget har det generiske grensesnittet `GenericDao<T>` og spesialiserte grensesnitt med JDBC-implementasjoner; tjenestene kjenner bare grensesnittene.
Utvikling	Maven-prosjekt med rotpakke `no.usn.booklibrary`. Produksjonskode i `src/main/java`, tester i `test/java` (konfigurert som `testSourceDirectory` i pom.xml), SQL-skript i `database/` og generert JavaDoc i `docs/`. Profilen `integration` kjører bare integrasjonstestene. Inngangspunkt: `no.usn.booklibrary.Launcher`.
Prosess	JavaFX-applikasjonstråden håndterer brukerhendelser. Hver databaseoperasjon åpner sin egen forbindelse via `DatabaseConnection` og lukker den med try-with-resources, slik at forbindelser ikke lekker ved unntak. **[fyll inn: eventuell bruk av bakgrunnstråder (Task)]**
Fysisk	Desktop-applikasjon på eierens maskin (JVM 17 med JavaFX 21) og en lokal MySQL 8-server. Tilkoblingsdata leses fra `~/.booklibrary/config.properties`, som opprettes ved første oppstart og ligger utenfor prosjektmappen.
Scenarier (+1)	UC2 «Lagre bok» (figur 3) og UC4 «Slette forfatter» (figur 4) går gjennom alle lag fra view til database. UC5 «Generere rapport» (figur 5) viser lesing med aggregering i databasen.

4 Databasedesign

4.1 ER-diagram

Databasen følger emnets foreslåtte modell (kapittel 8.2) med fire tabeller: `writer`, `genre`, `bookshelf` og `book`. Alle relasjoner er en-til-mange: en forfatter, en sjanger og en bokhylle kan ha null eller flere bøker, mens hver bok har nøyaktig én av hver.



Figur *7. ER-diagram for den implementerte databasen.*

Tidligere i prosjektet ble det skissert en større datamodell med flere tabeller. Den er lagt bort til fordel for oppgavens fire-tabellsmodell, som dekker alle kravene.

4.2 De fire hovedentitetene

Tabell *9. Hovedentiteter i databasen.*

Tabell	Innhold	Regler
writer	Forfattere: fornavn, etternavn, nasjonalitet, fødselsår	UNIQUE (first_name, last_name): samme forfatter kan ikke registreres to ganger
genre	Sjangre, f.eks. krim eller fantasy, med beskrivelse	UNIQUE name
bookshelf	Fysisk plassering: navn, rom og hyllenummer	Ingen unikhetsregel, siden to reoler kan ha samme navn
book	Tittel, utgivelsesår, ISBN, sidetall, lesestatus og fremmednøkler til writer, genre og bookshelf	UNIQUE isbn (valgfri, NULL tillatt); reading_status ENUM med standardverdi UNREAD; indekser på title, writer_id og genre_id

4.3 Integritetsregler og feilhåndtering

Integritetsreglene håndheves i databasen, og MySQL-feilkoder oversettes til domenespesifikke unntak i `JdbcSupport`, slik at brukeren får en forståelig melding:

Tabell *10. Kobling mellom databaseregler og unntak.*

Regel	MySQL-feil	Unntak i Java	Effekt for brukeren
UNIQUE-brudd (ISBN, forfatter, sjanger)	1062	DuplicateEntryException	«Finnes allerede»-melding
FK ON DELETE RESTRICT (FR12)	1451	EntityInUseException	Sletting hindres med forklaring
Ukjent fremmednøkkel	1452	DataAccessException	Lagring avvises
Øvrige SQL-feil / ingen forbindelse	—	DataAccessException	Generell feilmelding, ingen krasj

4.4 Testdata og testdatabase

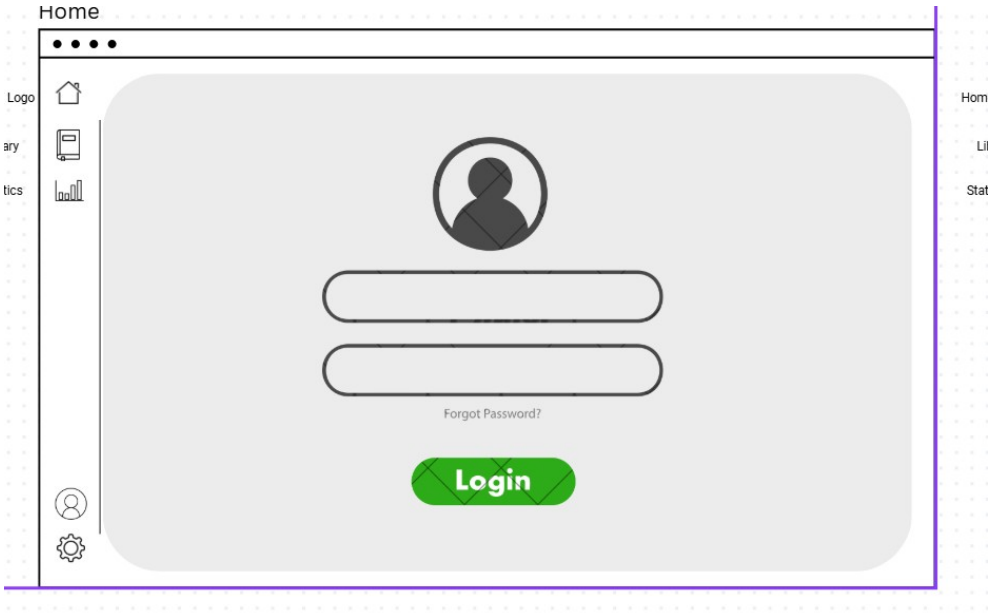
- `schema.sql` oppretter databasen og tabellene. `seed.sql` fyller inn 30 bøker (norske og internasjonale), 19 forfattere, 8 sjangre og 5 bokhyller. Én forfatter og én sjanger har bevisst ingen bøker, slik at rapporter og sletteskjerming kan demonstreres.
- Integrasjonstestene bruker en egen database (`booklibrary_test`) som opprettes med `test-setup.sql`. Testene nekter å kjøre hvis databasenavnet ikke slutter på `_test`, slik at den ordinære databasen aldri berøres.
- `database/README.md` dokumenterer ER-diagram, tabeller, regler og oppsett på Windows og Eclipse.

5 Brukergrensesnittdesign

5.1 Designprosess

UX/UI-arbeidet ledes av Laura. Prosessen har gått fra enkle wireframes med navigasjon, bokliste og søkefelt til mer detaljerte forslag med bokkort og fargepaletter. Wireframene er ment som retningslinjer for plassering og brukerflyt, ikke som ferdige skjermbilder.

Designet er forbedret gjennom tilbakemeldingsrunder i Teams. 24.09.2026 foreslo Eryk at grønt ikke burde være hovedfargen, at måneikonet var for skarpt og at knappen for å legge til bok burde flyttes lenger ned. Laura justerte knappeplasseringen 25.09.2026. Innloggingsskissen i de tidlige wireframene (figur 8) ble forkastet da løsningen ble forenklet til én bruker.



Figur *8. Tidlig wireframe med sidemeny (hjem, bibliotek, statistikk) og innloggingsskjerm. Innloggingen ble forkastet etter overgangen til én bruker.*

5.2 Implementert brukergrensesnitt (v0.1)

- **Hovedvindu** med boktabell, søkefelt og filtre for forfatter, sjanger, bokhylle og lesestatus (FR8–FR10).
- **Detaljpanel** som viser all informasjon om valgt bok (FR7).
- **Dialoger** for å legge til og redigere bøker, forfattere, sjangre og bokhyller, med validering og feilmeldinger via Alert (FR1–FR6).
- **Statuslinje** som viser totalt antall bøker og antall lest, leser nå og ulest (FR13).

[fyll inn: skjermbilder av implementert v0.1 – hovedvindu, bokdialog, detaljpanel og feilmelding ved sletteskjerming]

5.3 Navigasjonsflyt

Alle hovedfunksjoner nås fra hovedvinduet. Registrering og redigering skjer i egne dialoger, som lukkes tilbake til en oppdatert liste.

Tabell *11. Visninger og navigasjon.*

Visning	Funksjon	UC	Status
Hovedvindu (samling)	Bokliste, søk og filtre	UC1	Implementert
Detaljpanel	Detaljer om valgt bok	UC1	Implementert

Visning	Funksjon	UC	Status
Bokdialog	Legge til / redigere bok	UC2, UC3	Implementert
Administrasjonsdialoger	Forfattere, sjangre og bokhyller	UC4	Implementert
Statistikk	Diagrammer (PieChart/BarChart) over samlingen	UC5	Planlagt
Eksport	CSV-eksport av samling og rapport	UC6	Planlagt

5.4 Brukervennlighet og tilgjengelighet

Tabellen viser gruppens designmål for brukervennlighet og tilgjengelighet. De er mål for løsningen og er ikke verifisert; resultatene føres inn etter test.

Tabell *12. Designmål for brukervennlighet og tilgjengelighet.*

Område	Designmål	Verifisering
Kontrast	Tekst og knapper skal ha god kontrast mot bakgrunnen (mål: minst 4,5:1 for brødtekst, jf. WCAG 2.1 AA)	**[fyll inn: kontrollert med kontrastverktøy]**
Tastaturnavigasjon	Alle felt og knapper kan nås med Tab i logisk rekkefølge; Enter lagrer og Esc lukker dialoger	**[fyll inn: manuell test]**
Feilmeldinger	Tydelige Alert-dialoger som forklarer hva som er feil og hva brukeren kan gjøre, uten tekniske SQL-meldinger	Delvis oppfylt (AlertUtil, egne unntak); **[fyll inn]**
Lesbarhet	Lesbar skriftstørrelse og tydelige etiketter på alle felt og kolonner	**[fyll inn]**
Bekreftelse	Destruktive handlinger (sletting) krever bekreftelse	**[fyll inn: bekreft for alle entiteter]**
Konsistens	Samme plassering av knapper og samme dialogoppsett for alle entiteter	**[fyll inn]**

6 Implementasjonsoversikt

6.1 Krav mot implementasjon

Tabell *13. Sporbarhet fra krav til kode.*

Krav	Implementasjon (sentrale klasser)	Status
FR1–FR4	Controller- og view-klasser per entitet → BookService, WriterService, GenreService, BookshelfService → tilhørende DAO og DaoImpl	Implementert
FR5, FR6	Book med Writer, Genre, Bookshelf og ReadingStatus; FK-kolonner i book	Implementert
FR7	Detaljpanel i hovedvinduet	Implementert
FR8–FR10	SearchCriteria og BookDao.search(SearchCriteria) med tittel, writerId, genreId, bookshelfId og status	Implementert
FR11	ReportEntry, ReportDao, ReportDaoImpl, ReportService	Implementert (backend)
FR12	FK ON DELETE RESTRICT, JdbcSupport (1451 → EntityInUseException), AlertUtil	Implementert
FR13	Statuslinje i hovedvinduet	Implementert
FR14	Skal bygge på ReportService	Planlagt
NFR5	ConfigManager	Implementert

6.2 Viktige pakker og klasser

Tabell *14. Pakker og ansvar.*

Pakke	Sentrale klasser	Ansvar
model	Book, Writer, Genre, Bookshelf, ReadingStatus, SearchCriteria, ReportEntry	Domeneobjekter og filterkriterier
dao	GenericDao<T>, BookDao, WriterDao, GenreDao, BookshelfDao, ReportDao og *DaoImpl	All SQL; JDBC med PreparedStatement
service	BookService, WriterService, GenreService, BookshelfService, ReportService	Forretningsregler, validering og rapportlogikk
controller	Én controller per visning	Kobler brukerhendelser til tjenester
view	JavaFX-visninger og dialoger	Presentasjon
util	ConfigManager, DatabaseConnection, JdbcSupport, InputValidator, AlertUtil	Konfigurasjon, forbindelser, feilmapping, validering, dialoger
exception	DataAccessException, ValidationException, DuplicateEntryException, EntityInUseException, ConfigurationException	Egendefinerte unntak

6.3 Sentrale mekanismer

- **ConfigManager** oppretter `~/.booklibrary/config.properties` ved første oppstart (filsysteminteraksjon). Brukeren legger inn databasepassordet der, slik at påloggingsdata aldri havner i Git. Manglende eller ugyldig konfigurasjon gir `ConfigurationException`.
- **DatabaseConnection** åpner én forbindelse per operasjon i en try-with-resources-blokk. Det er enkelt og trygt for en enbrukerapplikasjon og hindrer lekkasje av forbindelser.
- **JdbcSupport** samler felles JDBC-hjelpefunksjoner og oversetter SQL-feilkoder til domeneunntak (tabell 10).
- **BookDao.search(SearchCriteria)** bygger en parametrisert spørring av de feltene som er satt. Jokertegn i LIKE-søk escapes, og integrasjonstestene verifiserer at søket er trygt mot SQL-injeksjon.

- **ReportDaoImpl** bruker `LEFT JOIN` og `COUNT(b.book_id)` i stedet for `COUNT(*)`, slik at en sjanger uten bøker gir 0 og ikke 1. `CONCAT_WS` gir lesbare hyllenavn også når rom eller hyllenummer mangler.

6.4 JavaDoc og kodedokumentasjon

Emnet krever at hver klasse oppgir forfatter(e), bidrag, hensikt og forklaring av metodene, og at generert JavaDoc leveres i `docs/`. Gruppen følger denne strukturen:

- Klassekommentar: hensikt, `@author` og en kort beskrivelse av hvem som har bidratt med hva.
- Metodekommentarer: beskrivelse, `@param`, `@return` og `@throws` for de egendefinerte unntakene.
- JavaDoc genereres med `mvn javadoc:javadoc` (kjører uten feil) og kopieres til `docs/` i ZIP-filen.

De nye klassene fra databasearbeidet (ReportDao, ReportDaoImpl, ReportService, ReportEntry og testklassene) er dokumentert på denne måten. **[fyll inn: bekreft at alle klasser har @author og bidragsbeskrivelse før innlevering]**

6.5 Bygg og kjøring

1. Opprett databasen: kjør `database/schema.sql` og deretter `database/seed.sql` i MySQL.
2. Importer i Eclipse: File → Import → Maven → Existing Maven Projects.
3. Kjør `no.usn.booklibrary.Launcher` én gang, legg inn passordet i `config.properties` og start på nytt.
4. Kjør testene med `mvn test` (se kapittel 7).

7 Testinnsikt

7.1 Teststrategi

- **Enhetstester** av tjenestelaget med falske DAO-er (f.eks. `FakeReportDao`). De krever ingen database og kjører på alle maskiner.
- **Integrasjonstester** av alle DAO-implementasjoner mot en ekte MySQL 8-database. Skjemaet bygges på nytt fra `schema.sql` før hver testklasse, og alle rader slettes før hver test.
- **Manuell funksjonstesting** av brukergrensesnittet etter en fast testliste (tabell 17).

Integrasjonstestene hoppes over automatisk (JUnit `Assumptions`) når ingen testdatabase er konfigurert, slik at `mvn test` alltid er grønn på maskiner uten MySQL.

7.2 Verktøy og miljø

Tabell *15. Testverktøy og miljø.*

Verktøy	Bruk
JUnit 5	Testrammeverk for enhets- og integrasjonstester, inkludert Assumptions
Maven Surefire	Kjører testene i `mvn test`; profilen `-P integration` kjører kun integrasjonstestene
MySQL 8.0.46	Testdatabasen `booklibrary_test`, satt opp med `test-setup.sql`
JDK 17	Kjøremiljø for testene

7.3 Resultater fra automatiske tester

Tabell *16. Automatiske tester per testklasse (kjøring mot MySQL 8.0.46 og JDK 17).*

Testklasse	Type	Antall	Resultat
BookDaoImplIntegrationTest	Integrasjon	16	Bestått
WriterDaoImplIntegrationTest	Integrasjon	10	Bestått
GenreDaoImplIntegrationTest	Integrasjon	6	Bestått
BookshelfDaoImplIntegrationTest	Integrasjon	4	Bestått
ReportDaoImplIntegrationTest	Integrasjon	3	Bestått
SeedDataIntegrationTest	Integrasjon	1	Bestått
ReportServiceTest (med FakeReportDao)	Enhet	8	Bestått
Øvrige enhetstester (bl.a. BookServiceTest, WriterServiceTest)	Enhet	13	Bestått
Totalt		**61**	**0 feil, 0 errors, 0 hoppet over**

Uten database kjører de 21 enhetstestene, og de 40 integrasjonstestene hoppes over. `mvn javadoc:javadoc` kjører også uten feil.

7.4 Hva testene dekker

- CRUD for alle fire entiteter (FR1–FR4), inkludert JOIN som fyller inn forfatter, sjanger og bokhylle på Book (FR5).
- Kolonner som kan være NULL (ISBN, nasjonalitet, rom, hyllenummer).
- UNIQUE-brudd som blir `DuplicateEntryException`, og FK-brudd ved sletting som blir `EntityInUseException` (FR12).
- Ukjente fremmednøkler ved lagring av bok.

- Søk på forfatter, bokhylle, sjanger og lesestatus (FR8–FR10), escaping av LIKE-jokertegn og sikkerhet mot SQL-injeksjon.
- Rapporter (FR11): riktige antall, grupper uten bøker gir 0, utfylling av alle lesestatuser og andel leste bøker.
- At seed-skriptet laster forventet antall rader.

7.5 Manuell funksjonstesting

Tabell *17. Manuelle funksjonstester av brukergrensesnittet.*

Test	Handling	Forventet resultat	Resultat
T01	Legg til bok med gyldige data	Boken lagres og vises i listen; statuslinjen oppdateres	[fyll inn]
T02	Lagre bok uten tittel / med ugyldig årstall	Valideringsmelding, ingen lagring	[fyll inn]
T03	Filtrer på forfatter, sjanger, bokhylle og status	Kun samsvarende bøker vises (DAO-søket er dekket av integrasjonstester)	[fyll inn]
T04	Endre lesestatus og start applikasjonen på nytt	Statusen er bevart	[fyll inn]
T05	Slett forfatter som har bøker	Sletting hindres med forståelig melding (dekket av integrasjonstest på DAO-nivå)	[fyll inn]
T06	Stopp MySQL-tjenesten og forsøk en operasjon	Forståelig feilmelding, ingen krasj	[fyll inn]
T07	Importer ZIP i Eclipse på minst to maskiner	Prosjektet bygger og starter uten tilpasning	[fyll inn]
T08	Generer JavaDoc	HTML-dokumentasjon i docs/	Bestått (mvn javadoc:javadoc)
T09	Naviger i hovedvindu og dialoger kun med tastatur	Alle felt og knapper kan nås (designmål, kap. 5.4)	[fyll inn]

7.6 Utfordringer i testarbeidet og løsninger

Tabell *18. Utfordringer i testarbeidet og løsninger.*

Utfordring	Løsning
Maven kjørte ingen tester, fordi testene lå i `test/java` og ikke i standardmappen `src/test/java`.	La inn `<testSourceDirectory>test/java</testSourceDirectory>` i pom.xml og en egen `integration`-profil.
To eksisterende testfiler kompilerte ikke, fordi en konstruktør i Writer var kommentert ut.	Gjeninnførte Writer-konstruktøren med fire argumenter; ingen annen eksisterende kode ble endret.
Risiko for at testene sletter data i den ordinære databasen.	Egen testdatabase; testene kjører bare når databasenavnet slutter på `_test`.
`COUNT(*)` med LEFT JOIN ga 1 for tomme grupper.	Bruker `COUNT(b.book_id)`, som hopper over NULL; dekket av egen test.
Testene måtte kunne kjøres uten MySQL.	JUnit Assumptions hopper over integrasjonstestene når ingen testkonfigurasjon finnes.

Testarbeidet påvirket designet: at databasen håndhever sletteskjermingen ble tydelig gjennom integrasjonstestene. Avhengigheten av DAO-grensesnitt gjorde det mulig å teste rapportlogikken uten database.

8 Utfordringer og begrensninger

8.1 Tekniske og konseptuelle utfordringer

- **Avgrensning av omfanget.** Tidlige skisser inneholdt flere funksjoner enn oppgaven krever, blant annet innlogging og en større datamodell. Etter råd fra faglærerne har gruppen avgrenset løsningen til oppgavens nøkkelfunksjoner og fire-tabellsmodellen.
- **Modeller i takt med koden.** Use case-diagrammet (v0.5) og klassediagrammet (v0.3) ble laget før koden. Klassediagrammet må oppdateres slik at det samsvarer med implementasjonen (kap. 3.2).
- **Testinfrastruktur.** Testene ble ikke kjørt av Maven før pom.xml ble rettet (kap. 7.6). Etter rettingen kjører alle 61 testene som en del av byggingen.
- **Integrasjon av bidrag.** Databasearbeidet ligger på en egen gren i en fork (`feature/database-integration-stian`) og leveres som pull request til organisasjonsrepositoriet når skrivetilgang er på plass.
- **[fyll inn: eventuelle utfordringer med JavaFX-oppsett i Eclipse eller andre tekniske problemer]**

8.2 Begrensninger i v0.1

- Rapportene (FR11) finnes i backend, men har ennå ingen egen statistikkvisning; statuslinjen viser totalt, lest, leser nå og ulest.
- CSV-eksport (FR14) er ikke implementert. Filsysteminteraksjonen i v0.1 er begrenset til konfigurasjonsfilen.
- Ingen brukertest med eksterne testpersoner er gjennomført ennå, og designmålene i kap. 5.4 er ikke verifisert.
- ZIP-import i Eclipse er ikke testet på flere maskiner ennå.
- Klassediagrammet (v0.3) samsvarer ikke helt med koden.

8.3 Neste steg og mulige utvidelser

Tabell *19. Plan for videre arbeid.*

Versjon	Innhold	Ansvarlig
v0.1.1 (nå)	FR1–FR10, FR12 og FR13 implementert; FR11 i backend; 61 automatiske tester	Eryk (kjerne), Stian (database, rapporter, tester)
v0.2 (før innlevering)	Statistikkvisning (PieChart/BarChart) basert på ReportService (FR11) og CSV-eksport (FR14)	Santheep, [fyll inn]

Mulige utvidelser utenfor oppgavens omfang, som ikke er planlagt i dette prosjektet:

- utlån og tilbakelevering av bøker
- CSV-import
- omslagsbilder
- lesemål per år

9 Individuelle bidrag

9.1 Arbeidsform

Arbeidet er organisert i 15 GitHub Issues: åtte for rapportkapitler og sju for funksjoner i applikasjonen. Hvert Issue har én ansvarlig, og Planner brukes til fordeling og oppfølging. Teams brukes til koordinering, deling av skisser og tilbakemeldinger, med egne kanaler for backend/database og UX/UI.

9.2 Bidragstabell

Tabell *20*. *Fordeling av ansvar og bidrag.*

Medlem	Rolle	Hovedbidrag	Rapport
Stian Petersen	Backend / database	Database og forenkling til én bruker; FR11-rapporter (ReportEntry, ReportDao, ReportDaoImpl, ReportService); 40 integrasjonstester og 8 enhetstester; retting av pom.xml slik at Maven kjører testene; seed.sql (30 bøker); database/README.md med ER-diagram; separat testdatabase	Kap. 4 og 7
Santheep [fyll inn]	Backend / database	Statistikkvisning (PieChart/BarChart) på ReportService og CSV-eksport (FR14) – planlagt; [fyll inn: utført arbeid]	[fyll inn]
Eryk [fyll inn]	Arkitektur og hovedutvikler	Arkitektur og lagdeling; initial implementasjon av v0.1 (størstedelen av koden), inkludert ConfigManager, DatabaseConnection og JdbcSupport; use case-diagram v0.5, klassediagram v0.3 og sekvensdiagram for sletting; tilbakemeldinger på design	[fyll inn]
Laura Gorska	UX/UI	Wireframes, brukerflyt og visuelle designforslag (fargepaletter, bokkort); oppfølging av designtilbakemeldinger	Kap. 5 [fyll inn]
Andreas [fyll inn]	Rapportansvarlig	Struktur og redigering av rapporten; [fyll inn]	Hele rapporten
Frank [fyll inn]	[fyll inn]	[fyll inn]	[fyll inn]

9.3 Individuelle refleksjoner

Hvert medlem skriver en kort refleksjon (3–5 setninger) om eget arbeid, ferdigheter og læringsutbytte.

Stian Petersen

Utkast: Gjennom databasearbeidet har jeg lært hvor mye det betyr å la databasen håndheve integritetsregler (UNIQUE og FK med RESTRICT) og å oversette feilkodene til tydelige unntak i Java. Integrasjonstester mot en ekte MySQL-database avdekket detaljer som enhetstester ikke fanger opp, for eksempel forskjellen på `COUNT(*)` og `COUNT(kolonne)` ved LEFT JOIN. Jeg har også erfart at testoppsettet må verifiseres: Maven kjørte ingen tester før pom.xml ble rettet. [fyll inn: Stian bekrefter eller justerer]

Santheep

[fyll inn]

Andreas

[fyll inn]

Eryk

[fyll inn]

Laura Gorska

[fyll inn]

Frank
[fyll inn]

10 Konklusjon

LibriKeep v0.1.1 er en kjørbar JavaFX-applikasjon som dekker nøkkelfunksjonene i oppgaven: registrering og vedlikehold av bøker, forfattere, sjangre og bokhyller, kobling av hver bok til forfatter, sjanger og bokhylle, lesestatus, detaljvisning, søk og visning av alle bøker per forfatter og per bokhylle. Oppsummeringsrapportene (FR11) er implementert og testet i backend, og 61 automatiske tester kjører uten feil.

Sammenhengen mellom analyse, design og implementasjon er tydelig. Use case-modellens «kontroller om enheten er i bruk» er realisert både i sekvensdiagrammet og som FK med RESTRICT og `EntityInUseException`, og domenemodellen speiler databaseskjemaet direkte. Lagdelingen med DAO-grensesnitt gir lav kobling og gjør tjenestene testbare uten database. Arbeidet har også vist at modellene må oppdateres i takt med koden, og at testoppsettet bør verifiseres tidlig.

Før innlevering prioriterer gruppen statistikkvisning og CSV-eksport (v0.2), oppdatert klassediagram, skjermbilder av den implementerte løsningen, manuelle tester inkludert ZIP-import på flere maskiner, og en brukermanual. Mulige fremtidige utvidelser kan for eksempel være utlån, lesemål eller enkle AI-baserte anbefalinger, men disse ligger utenfor oppgavens omfang.

Vedlegg A – Sjekkliste før innlevering

Tabell *21. Sjekkliste før innlevering 11.11.2026 kl. 23:59.*

#	Punkt	Ansvarlig	Status
1	Oppdatere FR-ID-er og ansvarlige i GitHub Issues etter ny nummerering	[fyll inn]	Åpen
2	Oppdatere klassediagram (v0.4) slik at det samsvarer med koden	Eryk [fyll inn]	Åpen
3	Kontrollere klassenavn i sekvensdiagrammene mot koden	[fyll inn]	Åpen
4	Flette inn pull request for database og tester i hovedrepositoriet	Stian / Eryk	Venter på skrivetilgang
5	Implementere statistikkvisning (FR11) og CSV-eksport (FR14)	Santheep	Planlagt
6	Legge inn skjermbilder av implementert applikasjon (kap. 5)	Laura [fyll inn]	Åpen
7	Gjennomføre manuelle tester T01–T09 og føre inn resultater	[fyll inn]	Åpen
8	Teste ZIP-import i Eclipse på minst to maskiner	[fyll inn]	Åpen
9	Generere JavaDoc til docs/ og kontrollere @author og bidrag i alle klasser	Alle	Delvis
10	Skrive brukermanual og legge den i ZIP-filen (kap. 7.3.1 i emnebeskrivelsen)	[fyll inn]	Åpen
11	Legge ved SQL-skript (schema.sql, seed.sql) i ZIP-filen	Stian	Klar
12	Fylle inn individuelle bidrag og refleksjoner (kap. 9)	Alle	Åpen
13	Kontrollere siste beskjed i Canvas og at rapporten er under 40 sider	Andreas	Åpen
14	Eksportere rapporten til PDF og pakke ZIP	Andreas	Åpen

Vedlegg B – Sporbarhet mot emnets krav

Tabell *22*. *Vurderingskriterier for den skriftlige delen og hvor de dokumenteres.*

Område	Poeng	Dokumentert i
Funksjoner og kompleksitet	40	Kap. 2.3 (FR), 6.1 (krav mot kode), 7 (testresultater)
Programvaredesign	25	Kap. 3 (use case, klasser, sekvens, 4+1) og 4 (database)
UX/UI-design	20	Kap. 5 (designprosess, implementert UI, navigasjon, brukervennlighet)
Rapportens innhold og struktur	15	Følger emnets kapittel 7.6; kap. 9 (individuelle bidrag)

Tabell *23*. *Kontroll mot minimums- og spesifikke krav (emnebeskrivelsen 7.2–7.3).*

Krav i emnebeskrivelsen	Status	Referanse
Hovedvisning som gir tilgang til funksjonene	Oppfylt	Kap. 5.2
Tilkobling og tilgang til MySQL	Oppfylt	Kap. 4, NFR2
Generert JavaDoc	Oppfylt (genereres); docs/ i ZIP gjenstår	Kap. 6.4, NFR6
Forfatter, bidrag, hensikt og metoder i hver klasse	Delvis	NFR7
Java SE, JavaFX/Swing, Maven	Oppfylt	NFR1, NFR3
Eclipse-ZIP som kan importeres på rotnivå	Ikke verifisert	NFR4, T07
SQL-skript for oppretting og testdata (egen database)	Oppfylt	Kap. 4.4
Genererer ressursfiler som mangler	Oppfylt	NFR5
Interaksjon med filsystemet	Delvis (konfigurasjonsfil); CSV-eksport planlagt	FR14
Kjøretidsfeil håndteres	Oppfylt for validering og database; manuell test gjenstår	NFR8
Klassemedlemmer samsvarer med klassediagrammet	Ikke oppfylt ennå	Kap. 3.2
Brukermanual i ZIP	Ikke startet	Vedlegg A

Kilde: OAP200V – Course overview, høst 2026 (Mangaroska og Vesin), kapittel 7–8.